

Power Analysis of Scan and Non-Scan Flip-Flop Architectures

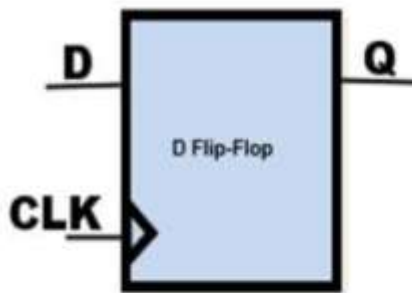
OBJECTIVE:

The main objective of this project is to design and compare conventional and scan-based 8-bit D Flip-Flop registers at the RTL level.

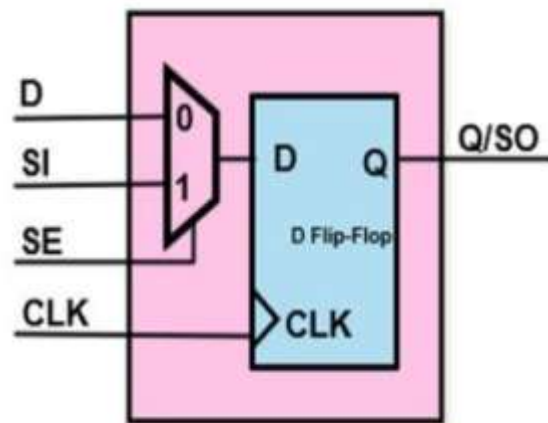
CONCEPT:

D Flip-Flop (Data Flip-Flop):

A D Flip-Flop is a basic memory element used in digital circuits to store one bit of data (0 or 1). It has two main inputs: D (Data input), Clock (CLK) and one output Q (Stored value). The value at D is copied to Q only when the clock edge occurs (usually rising edge). After that, the value is held (stored) until the next clock edge.



D Flip-Flop



Scan-Based D Flip-Flop

Scan-Based D Flip-Flop:

A Scan D Flip-Flop is an improved version of the normal D Flip-Flop, designed for easy testing of circuits. It is widely used in Design for Testability (DFT). In addition to the normal inputs, it includes two extra inputs: Scan Input (SI) and Scan Enable (SE). A 2:1 Multiplexer (MUX) is added at the input of the flip-flop. The MUX selects between the normal data input (D) and the scan input (SI) based on the value of the scan enable signal (SE).

The Scan D Flip-Flop operates in two modes:

1. Normal Mode ($SE = 0$):

- The MUX selects the D input
- The flip-flop behaves like a regular D Flip-Flop
- Data is captured on the clock edge and stored at the output

2. Scan Mode ($SE = 1$):

- The MUX selects the Scan Input (SI)
- Flip-flops are connected in a scan chain, forming a shift register
- Test data is shifted into the circuit through SI and shifted out through the output
- This allows internal states of the circuit to be easily controlled and observed

DESIGN FOR TESTABILITY (DFT):

Design for Testability (DFT) is a technique used in digital circuit design to make a system easy to test for faults after manufacturing.

Controllability (in DFT): The ability to easily force internal circuit nodes to a desired value (0 or 1) using test inputs (e.g., via scan chains).

Observability (in DFT): The ability to easily observe internal node values at the output pins (e.g., through scan output) for fault detection.

Stuck-at-0 fault: A node is permanently fixed at logic 0 regardless of the input.

Stuck-at-1 fault: A node is permanently fixed at logic 1 regardless of the input.

Original Circuit:

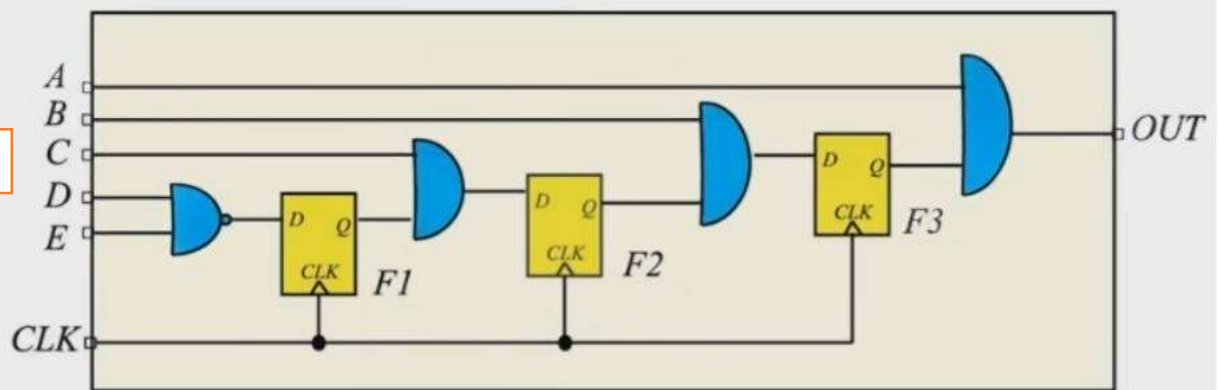
We have 3 flip-flops $\rightarrow F1, F2, F3$. They are part of a sequential circuit (like a counter). If we want to set: $F1 = 1, F2 = 1, F3 = 1$ (state = 111). We cannot directly set them, we must apply inputs, wait for clock cycles and let values propagate step by step. Thus for 3 flip-flops: $2^3 = 8$ clock cycles (worst case).

Modified Circuit:

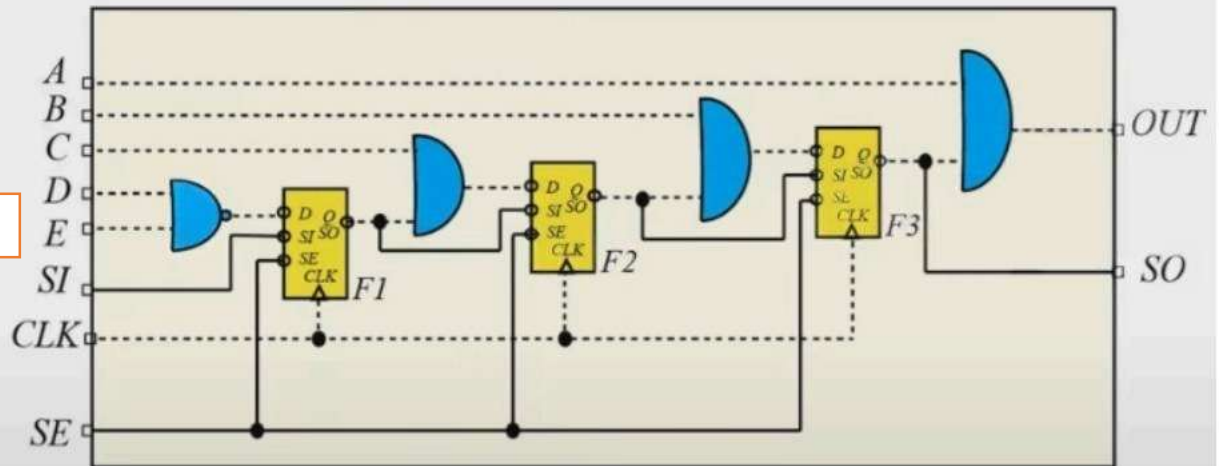
Each flip-flop is replaced with Scan Flip-Flop. They are connected as a chain $SI \rightarrow F1 \rightarrow F2 \rightarrow F3 \rightarrow SO$.

$F1 = 1, F2 = 0, F3 = 1$. Shift Mode ($SE = 1$) now circuit acts like a shift register. $SI = 1 \rightarrow 0 \rightarrow 1$ (one per clock).

ORIGINAL



MODIFIED



Clock 1:

Input: 1

F1 = 1

F2 = 0

F3 = 0

Clock 2:

Input: 0

Now shifting happens:

F1 = 0

F2 = 1

F3 = 0

Clock 3:

Input: 1

$F1 = 1$

$F2 = 0$

$F3 = 1$

Final Result:

$F1 = 1, F2 = 0, F3 = 1$

Done in 3 clock cycles.

Shift → Capture → Shift Cycle:

SHIFT (Load Test Data)

Set $SE = 1$ (Scan mode ON)

Use SI (Scan Input) to load values into flip-flops

Flip-flops act like a shift register

Data moves like:

$SI \rightarrow F1 \rightarrow F2 \rightarrow F3$

$F1 = 1, F2 = 0, F3 = 1$

Apply: $1 \rightarrow 0 \rightarrow 1$ through SI

Takes 3 clock cycles

Now internal states are fully controlled

CAPTURE (Run Circuit)

Set $SE = 0$ (Normal mode)

Give 1 clock pulse

Flip-flops stop shifting

Circuit works normally

Logic processes inputs

Output of logic goes into D of flip-flops, that value gets captured into Q.

Example (Fault detection)

We expect output = 1, but there is a fault (stuck-at-0) actual output becomes 0.

During capture: This wrong value (0) gets stored in flip-flop

SHIFT (Read Result)

Set SE = 1 again

Start clocking

Stored values shift out through SO (Scan Output)

SO $\rightarrow$ 0 (faulty value).

RTL CODING:

NON-SCAN:

VERILOG:

```
module dff_8bit (
    input clk,
    input rstn,          // Active LOW reset (aligned with your counter style)
    input [7:0] d,
    output reg [7:0] q
);
always @(posedge clk) begin
    if (!rstn)
        q <= 8'b00000000;
    else
        q <= d;
end
endmodule
```

TEST BENCH:

```

module tb_dff_8bit;
reg clk;
reg rstn;
reg [7:0] d;
wire [7:0] q;
dff_8bit uut (
    .clk(clk),
    .rstn(rstn),
    .d(d),
    .q(q)
);
// Clock generation (100 MHz)
always #5 clk = ~clk;
initial begin
    $dumpfile("dff8_activity.vcd");
    $dumpvars(0, tb_dff_8bit);
    // Initialize
    clk = 0;
    rstn = 0;
    d = 8'b00000000;
    // Release reset
    #20 rstn = 1;
    // Stimulus
    #10 d = 8'b10101010;
    #10 d = 8'b11110000;
    #10 d = 8'b00001111;
    #10 d = 8'b11001100;
    // Toggle reset again (to increase switching activity)
    #20 rstn = 0;
    #10 rstn = 1;

    #30 $finish;
end
endmodule

```

SCAN:**VERILOG:**

```

module scan_reg_8bit (
    input clk,
    input rstn,          // Active LOW reset
    input [7:0] d,
    input scan_in,
    input scan_en,
    output reg [7:0] q,
    output scan_out
);
integer i;
always @(posedge clk) begin
    if (!rstn)
        q <= 8'b00000000;
    else if (scan_en) begin
        q[0] <= scan_in;
        for (i = 1; i < 8; i = i + 1)
            q[i] <= q[i-1];
    end
    else
        q <= d;
end
assign scan_out = q[7];
endmodule

```

TEST BENCH:

```

module tb_scan_reg;
reg clk;
reg rstn;
reg [7:0] d;
reg scan_in, scan_en;
wire [7:0] q;
wire scan_out;
scan_reg_8bit uut (
    .clk(clk),
    .rstn(rstn),
    .d(d),
    .scan_in(scan_in),
    .scan_en(scan_en),
    .q(q),
    .scan_out(scan_out)
);
// Clock
always #5 clk = ~clk;
initial begin

```

```

$dumpfile("scan_activity.vcd");
$dumpvars(0, tb_scan_reg);
clk = 0;
rstn = 0;
scan_en = 0;
scan_in = 0;
d = 8'b00000000;
#20 rstn = 1;
// Functional Mode
scan_en = 0;
#10 d = 8'b11001100;
#10 d = 8'b10101010;
// Scan Mode
scan_en = 1;
scan_in = 1; #10;
scan_in = 0; #10;
scan_in = 1; #10;
scan_in = 1; #10;
scan_in = 0; #10;
scan_in = 0; #10;
scan_in = 1; #10;
scan_in = 0; #10;
// Back to normal mode
scan_en = 0;
#20 d = 8'b11110000;
// Reset again (extra switching)
#20 rstn = 0;
#10 rstn = 1;
#30 $finish;
end
endmodule

```

FUNCTIONAL SIMULATION AND VERIFICATION:

Functional simulation and verification were performed using testbenches for both conventional and scan-based 8-bit registers. A clock signal with 10 ns period and reset were used to initialize and control the design. For the non-scan design, different input patterns were applied and the output was verified to follow the input correctly on each clock edge. Reset functionality was also validated. For the scan-based design, both functional mode and scan mode were tested using scan_en and scan_in signals. The scan chain operation was verified by shifting data and observing correct output at scan_out.

SIMULATION OUTPUTS:**Non-Scan Flip-Flop:****AREA:**

Applications Places Text Editor

Open

constraints_sdc_scan.sdc × rcscript_scan.tcl × ctr_area_scan.rep × ctr_ne

```

=====
Generated by:      Genus(TM) Synthesis Solution 21.14-s082_1
Generated on:      Mar 20 2026 02:30:55 am
Module:           dff_8bit
Operating conditions: fast (balanced_tree)
Wireload mode:    enclosed
Area mode:        timing library
=====

```

Instance	Module	Cell Count	Cell Area	Net Area	Total Area	Wireload
dff_8bit		8	163.490	0.000	163.490	<none> (D)

(D) = wireload is default in technology library

POWER:

nonscan.v × constraints_sdc_nonscan.sdc

Instance: /dff_8bit
Power Unit: W
PDB Frames: /stim#0/frame#0

Category	Leakage	Internal	Switching	Total	Row%
memory	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.00%
register	1.80426e-06	2.43047e-05	2.76142e-05	5.37232e-05	97.07%
latch	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.00%
logic	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.00%
bbox	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.00%
clock	0.000000e+00	0.000000e+00	1.62342e-06	1.62342e-06	2.93%
pad	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.00%
pm	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.00%
Subtotal	1.80426e-06	2.43047e-05	2.92376e-05	5.53466e-05	100.00%
Percentage	3.26%	43.91%	52.83%	100.00%	100.00%

DELAY:

```
=====
Generated by:      Genus(TM) Synthesis Solution 21.14-s082_1
Generated on:      Mar 20 2026  02:30:55 am
Module:            dff_8bit
Operating conditions: fast (balanced_tree)
Wireload mode:     enclosed
Area mode:         timing library
=====

Path 1: MET (1088 ps) Setup Check with Pin q_reg[1]/CK->D
  Group: clk
  Startpoint: (R) rstn
  Clock: (R) clk
  Endpoint: (R) q_reg[1]/D
  Clock: (R) clk

      Capture      Launch
      Clock Edge:+ 2000      0
      Drv Adjust:+  0        0
      Src Latency:+ 0        0
      Net Latency:+ 0 (I)    0 (I)
      Arrival:=    2000      0

      Setup:-      102
      Uncertainty:- 10
      Required Time:= 1888
      Launch Clock:- 0
      Input Delay:-  800
      Data Path:-    0
      Slack:=      1088

Exceptions/Constraints:
input_delay      800      constraints_sdc_nons_line_7

#-----
# Timing Point  Flags  Arc  Edge  Cell      Fanout Load Trans Delay Arrival Instance
#                                     (fF) (ps) (ps) (ps) Location
#-----
# rstn          -      -    R    (arrival)  8 14.4  120  0    800  (-,-)
# q_reg[1]/D    <<<    -    R    DFFTRXL   8  -    -    0    800  (-,-)
#-----
```

Scan Flip-Flop:

AREA:

ApplicationsPlacesText Editor

Open

constraints_sdc_scan.sdc ×

rcscript_scan.tcl ×

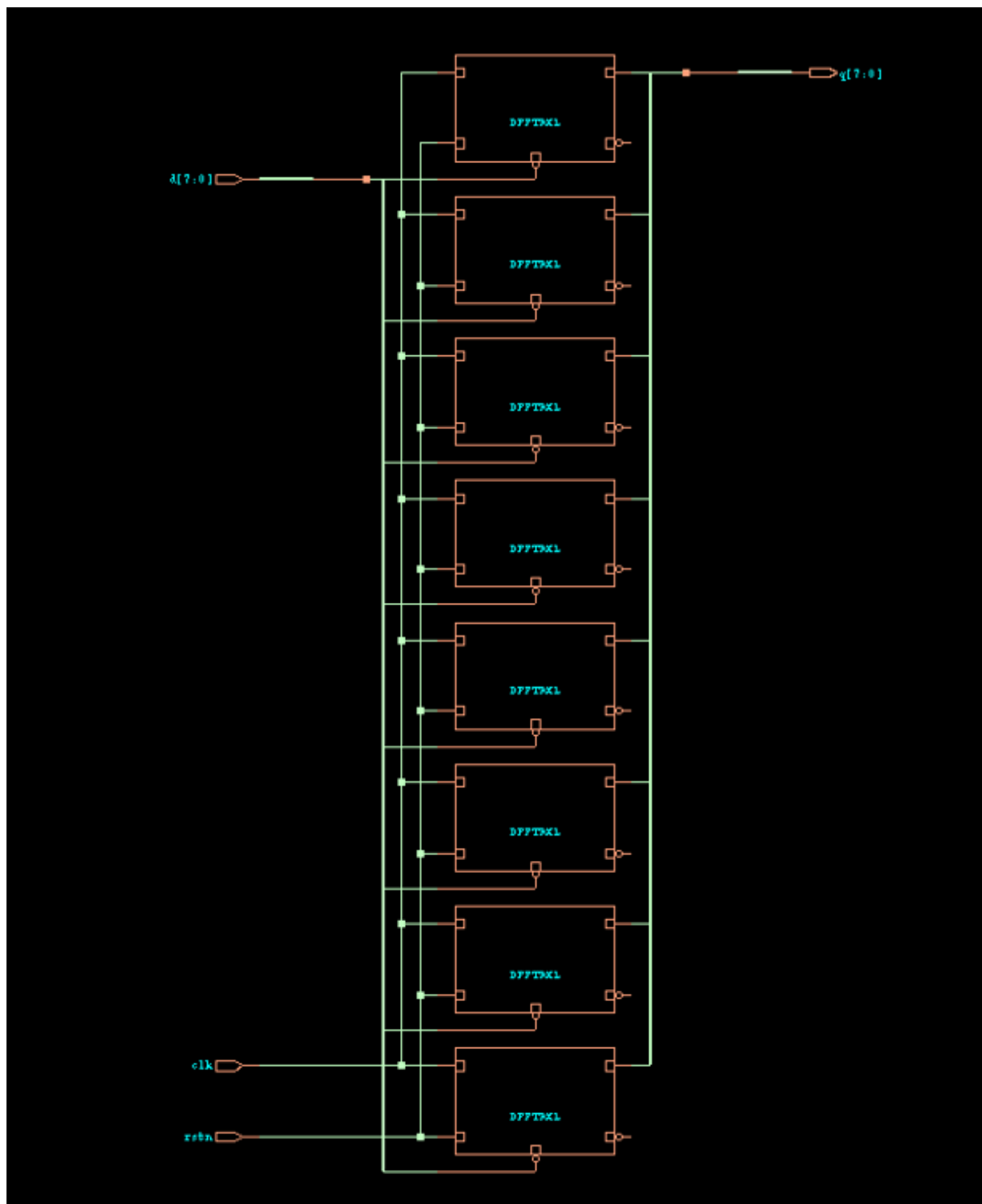
ctr_area_scan.rep ×

ctr_netlist_scan...

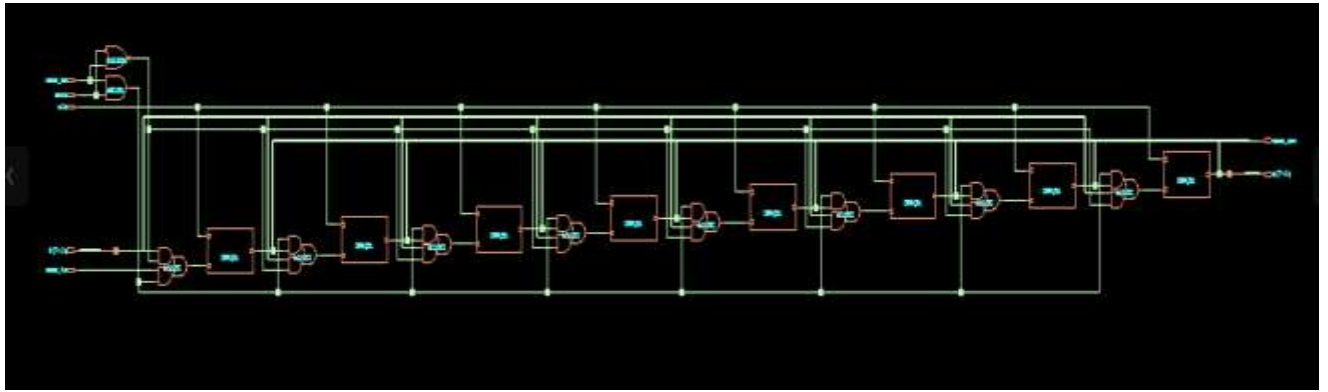
```
=====
Generated by:      Genus(TM) Synthesis Solution 21.14-s082_1
Generated on:      Mar 20 2026  02:42:42 am
Module:            scan_reg_8bit
Operating conditions: fast (balanced_tree)
Wireload mode:     enclosed
Area mode:         timing library
=====

Instance  Module  Cell Count  Cell Area  Net Area  Total Area  Wireload
-----
scan_reg_8bit      18    196.794    0.000    196.794 <none> (D)
(D) = wireload is default in technology library
```

#	Timing Point	Flags	Arc	Edge	Cell	Fanout	Load (fF)	Trans (ps)	Delay (ps)	Arrival (ps)	Instance Location
#	rstn	-	-	R	(arrival)	2	3.6	120	0	800	(-, -)
#	g309_2802/Y	-	AN->Y	R	N0R2BX1	8	14.4	87	76	876	(-, -)
#	g305_6783/Y	-	B1->Y	R	A022XL	1	1.7	28	36	912	(-, -)
#	q_reg[1]/D	<<<	-	R	DFFQX1	1	-	-	0	912	(-, -)

SCHEMATIC:Non-Scan Flip-Flop:

Scan Flip-Flop:



Observations:

Flip-Flop Design	Power	Area	Delay
Non-Scan	5.53466e-05	163.490	Data Path=0ps Setup=102ps Delay=102ps
Scan	6.23422e-05	196.794	Data Path=112ps Setup=99ps Delay=211ps

PERFORMANCE METRICS:

The performance of both non-scan and scan-based flip-flop designs was evaluated in terms of power, area, and delay using synthesis reports. The non-scan design shows a total power dissipation of 5.53466e-05, while the scan design shows a higher power dissipation of 6.23422e-05, due to additional switching activity and leakage introduced by scan circuitry. The area of the scan design (196.794) is greater than the non-scan design (163.490) because of the inclusion of multiplexers and extra scan logic. In terms of delay, the non-scan design has a delay of 102 ps, whereas the scan design has a higher delay of 211 ps, mainly due to the additional data path introduced by scan elements. Thus, the scan-based design increases power dissipation, area, and delay, but improves the Error Metric Evaluation

ERROR METRIC EVALUATION:

Error metrics are used to evaluate the accuracy and reliability of the design by comparing the expected output with the observed output during simulation. In this project, functional verification was performed using test cases, and the outputs

were analyzed to detect any mismatches. For both non-scan and scan-based designs, the simulated outputs matched the expected results for all applied test patterns, indicating zero functional error. Any mismatch between expected and actual output would indicate the presence of faults such as stuck-at faults or incorrect logic behavior. Thus, the error metric in this case is based on output comparison, and since no discrepancies were observed, the designs are considered functionally correct and error-free. stability of the circuit.

COMPARATIVE ANALYSIS:

A comparison was carried out between the conventional non-scan flip-flop design and the scan-based flip-flop design based on power, area, delay, and testability. The non-scan design shows lower power dissipation, smaller area, and reduced delay, making it more efficient in terms of performance. In contrast, the scan-based design exhibits higher power dissipation, larger area, and increased delay due to the inclusion of additional scan logic and multiplexers. However, the scan-based design provides significant advantages in terms of testability, allowing easier fault detection and improved controllability and observability of internal nodes. While the non-scan design is more optimal for performance, the scan-based design is preferred in practical VLSI systems due to its enhanced testing capability.

CONCLUSION:

The scan-based flip-flop design increases power, area, and delay compared to the non-scan design due to additional scan circuitry. However, it significantly improves testability, making it more suitable for reliable VLSI system design. Thus, the implementation was successfully completed and the analysis was carried out effectively.